
Exercises

Advanced Lesson 2

Your Name: Mikey Ferguson

Your Math Class: MATH 7013 - Advanced Mathematical Modeling

Remember to clear all variables that you will use below: `Clear[a, b, c, d, i, n, f, g, h]`

`Clear[a, b, c, d, i, n, f, g, h];`

1: After evaluation, how many digits of numerical accuracy do each of the following expressions have?

- $3/7$
- $2+3/7$
- $2.0+3.0/7.0$ (Hint: You can use `InputForm[2.0 + 3.0/7.0]` to see all digits.)
- $1.23+1.2345/1.234567$ (See the Hint.)
- $1.2+1.2345678901234567890+1.23456789012345678901234567890$
- $1.2345678901234567890+1.23456789012345678901234567890$
- $N[2/7]$ (See the Hint.)
- $N[2/7,100]$
- After evaluation, why does *Mathematica* leave $\sqrt{2}$ as $\sqrt{2}$ and not as $\sqrt{2} \rightarrow$

1.4142135623730951

$\left\{\frac{3}{7}, \text{Precision}\left[\frac{3}{7}\right]\right\}$

$\left\{2 + \frac{3}{7}, \text{Precision}\left[2 + \frac{3}{7}\right]\right\}$

$\left\{\text{InputForm}\left[2.\` + \frac{3.\`}{7.\`}\right], \text{Precision}\left[2.\` + \frac{3.\`}{7.\`}\right]\right\}$

$\left\{\text{InputForm}\left[1.23\` + \frac{1.2345\`}{1.234567\`}\right], \text{Precision}\left[1.23\` + \frac{1.2345\`}{1.234567\`}\right]\right\}$

$\left\{\text{InputForm}\left[1.2\` + 1.234567890123456789\`19.0915149772127 + 1.2345678901234567890123456789\`29.091514977212704\right], \text{Precision}\left[1.2\` + 1.234567890123456789\`19.0915149772127 + 1.2345678901234567890123456789\`29.091514977212704\right]\right\}$

$\left\{\text{InputForm}\left[1.234567890123456789\`19.0915149772127 + 1.2345678901234567890123456789\`29.091514977212704\right], \text{Precision}\left[1.234567890123456789\`19.0915149772127 + 1.2345678901234567890123456789\`29.091514977212704\right]\right\}$

$$\{\text{InputForm}\left[N\left[\frac{2}{7}\right]\right], \text{Precision}\left[N\left[\frac{2}{7}\right]\right]\}$$

$$\left\{N\left[\frac{2}{7}, 100\right], \text{Precision}\left[N\left[\frac{2}{7}, 100\right]\right]\right\}$$

$$\{\sqrt{2}, N[\sqrt{2}]\}$$

Exact numbers carry infinite precision, so $3/7$ and $2 + 3/7$ stay as fractions. A number written with a decimal point is a machine number carrying about 16 digits, so $2.0 + 3.0/7.0$ and $1.23 + 1.2345/1.234567$ have about 16. When precisions are mixed the answer is only as good as the worst input: adding 1.2 to the 19-digit and 29-digit numbers drops the result to machine precision, while the 19- and 29-digit numbers on their own give 19. $N[2/7]$ gives about 16 digits and $N[2/7, 100]$ gives 100. $\text{Sqrt}[2]$ stays as $\text{Sqrt}[2]$ because it is exact; Mathematica only turns it into a decimal when asked with N .

2: ● i: Evaluate exactly: $\sum_{n=1}^{100} \frac{1}{n}$

● ii: Evaluate: $\sum_{n=1}^{100} \frac{1.0}{n}$

● iii: Evaluate: $\sum_{n=1}^{100} N\left[\frac{1}{n}\right]$

● iv: Evaluate: $N\left[\sum_{n=1}^{100} \frac{1}{n}\right]$

● v: Evaluate: $\sum_{n=1}^{100} \frac{1}{n}$ to 50 digit accuracy

● vi: Try to evaluate: $\sum_{n=1}^{100} \frac{1.0}{n}$ to 50 digit accuracy using $N\left[\sum_{n=1}^{100} \frac{1.0}{n}, 50\right]$

(Keep the 1.0)

(Why did it fail to give 50 digit accuracy?)

$$\sum_{n=1}^{100} \frac{1}{n}$$

$$\sum_{n=1}^{100} \frac{1.}{n}$$

$$\sum_{n=1}^{100} N\left[\frac{1}{n}\right]$$

$$N\left[\sum_{n=1}^{100} \frac{1}{n}\right]$$

$$N\left[\sum_{n=1}^{100} \frac{1}{n}, 50\right]$$

$$N\left[\sum_{n=1}^{100} \frac{1.}{n}, 50\right]$$

All six are the same sum. Written exactly it is one large fraction. Writing $1.0/n$ makes every term approximate, so that version carries only about 16 digits. Asking for 50 digits works from the exact sum, because Mathematica can go back to the fraction and recompute. It fails for the 1.0 version: those digits never existed, and N cannot invent them.

3: ● Evaluate to 500 significant digits: e^π (Remember to use e from the palette and not e)

● Evaluate exactly: e^π Why did it remain the same?

$N[e^\pi, 500]$

e^π

$N[E^\pi, 500]$ prints 500 digits. Evaluated on its own, E^π stays as E^π because it is exact, and Mathematica never converts an exact expression to a decimal unless asked.

4: ● Evaluate the complex number exactly: $\sum_{n=1}^{10} \frac{1}{n+2i}$

(Use the Basic Math Input Palette to enter " i ".)

● Evaluate to 50 significant digits: $\sum_{n=1}^{10} \frac{1}{n+2i}$

● Try to evaluate $\sum_{n=1}^{10} \frac{1}{n+2.0i}$ to 50 significant digits

(Keep the 2.0)

(Why did it fail to give 50 digit accuracy?)

$$\sum_{n=1}^{10} \frac{1}{n+2i}$$

$$N\left[\sum_{n=1}^{10} \frac{1}{n+2i}, 50\right]$$

$$N\left[\sum_{n=1}^{10} \frac{1}{n+2.0i}, 50\right]$$

The exact sum is a single complex fraction, and N of it prints 50 digits. Writing 2.0 I instead of 2 I makes every term approximate, so the answer stops at about 16 digits no matter how many are requested.

5: ● Evaluate exactly: **Sin[6]**

● Evaluate: **Sin[6.0]**

● Evaluate to 50 significant digits: **Sin[6]**

● Try to evaluate to 50 significant digits: **Sin[6.0]** Why does it fail to get 50 digits?

● Try to evaluate to 50 significant digits: **Sin[N[6, 20]]** Why does it fail to get 50 digits?

Sin[6]

Sin[6.]

N[Sin[6], 50]

N[Sin[6.], 50]

`N[Sin[N[6, 20]], 50]`

`Sin[6]` stays as `Sin[6]`: 6 is exact and the sine has no simpler exact form. `Sin[6.0]` is -0.279415. `N[Sin[6], 50]` gives 50 correct digits because it starts from the exact form. `N[Sin[6.0], 50]` cannot, because 6.0 carries only about 16 digits, and `N[Sin[N[6, 20]], 50]` stops near 19 digits for the same reason. An answer can never be more precise than its input.

6: ● Define a user function: **`f[nTerms_]`** using the formula $\sum_{n=1}^{nTerms} \frac{1}{n}$ which will evaluate exactly.

● Evaluate: **`f[1]`**, **`f[10]`**, **`f[100]`**

`Clear[f, n, nTerms];`

`f[nTerms_] := Sum[1/n, {n, 1, nTerms}]`

`{f[1], f[10], f[100]}`

7: It is possible to determine the name associated with every system function using:

**`Names["System`*"]`** (Remember the "wild card" \* and use ` and not ')

● Evaluate the above expression to determine which system functions are available.

Please **delete** this list of system functions **before** you print your notebook because the list is too long to print!

● Determine the total number of system functions available:

`Length[Names["System`*"]]`

`Names["System`*"]`

`Length[Names["System`*"]]`

8: ● Part 1: Define a function (name it **`myDiff1[...]`**) that differentiates a function **`f`** with respect to **`x`** using **immediate** assignment (i.e. "=")

`Clear[x, f];`

`myDiff1[f_] = D[f, x]`

● Part 2: Evaluate the following lines of code:

`myDiff1[x^2 + 3 x + 4]`

`myDiff1[e^(2 x + 3)]`

`myDiff1[f[x]]`

`? myDiff1`

Explain (in a Text Cell) why you did NOT get the correct answers for differentiation.

- Part 3: Define a function (name it **myDiff2[...]**) that differentiates a function **f** with respect to **x** using delayed assignment (i.e. ":=")

myDiff2[f_] := $\partial_x f$

- Part 4: Evaluate the following lines of code:

myDiff2[$x^2 + 3x + 4$]

myDiff2[e^{2x+3}]

myDiff2[f[x]]

? myDiff2

Explain (in a Text Cell) why you DID get the correct answers.

- Part 5: Explain (in a Text Cell) why you must (for this case) use delayed assignment (i.e. ":=") to define your function **myDiff2[...]** so that it will work properly.

```
Clear[x, f];
```

```
myDiff1[f_] =  $\partial_x f$ 
```

```
{myDiff1[ $x^2 + 3x + 4$ ], myDiff1[ $e^{2x} + 3$ ], myDiff1[f[x]]}
```

```
?myDiff1
```

```
Clear[x, f];
```

```
myDiff2[f_] :=  $\partial_x f$ 
```

```
{myDiff2[ $x^2 + 3x + 4$ ], myDiff2[ $e^{2x} + 3$ ], myDiff2[f[x]]}
```

```
?myDiff2
```

Part 2: myDiff1 was defined with =, so the right-hand side was worked out once, at definition time, when f was still an empty symbol. The derivative of the symbol f with respect to x is 0, so 0 is what got stored, and myDiff1 returns 0 for every input. Part 4: myDiff2 was defined with :=, which stores the right-hand side unevaluated. The derivative is taken when the function is called, on the expression actually passed in, so the answers are correct. Part 5: whenever the right-hand side only makes sense once the argument has a value, the definition must be delayed. Immediate assignment computes on an empty symbol and freezes that answer.